Secure Software Development Life Cycle (SSDLC)

- **Project Name**: LibOrate
- **Organization**: AImpower.org
- **Last Reviewed**: July 2025
- **Owner**: AImpower.org Security & Dev Team

1. Introduction

LibOrate is a videoconferencing companion Zoom App designed to support individuals who stutter by offering real-time emotional and relational support during videoconferencing. We adopt a Secure Software Development Lifecycle (SSDLC) model to embed security across all stages of development.

This document outlines the security practices and evidence associated with our SSDLC.

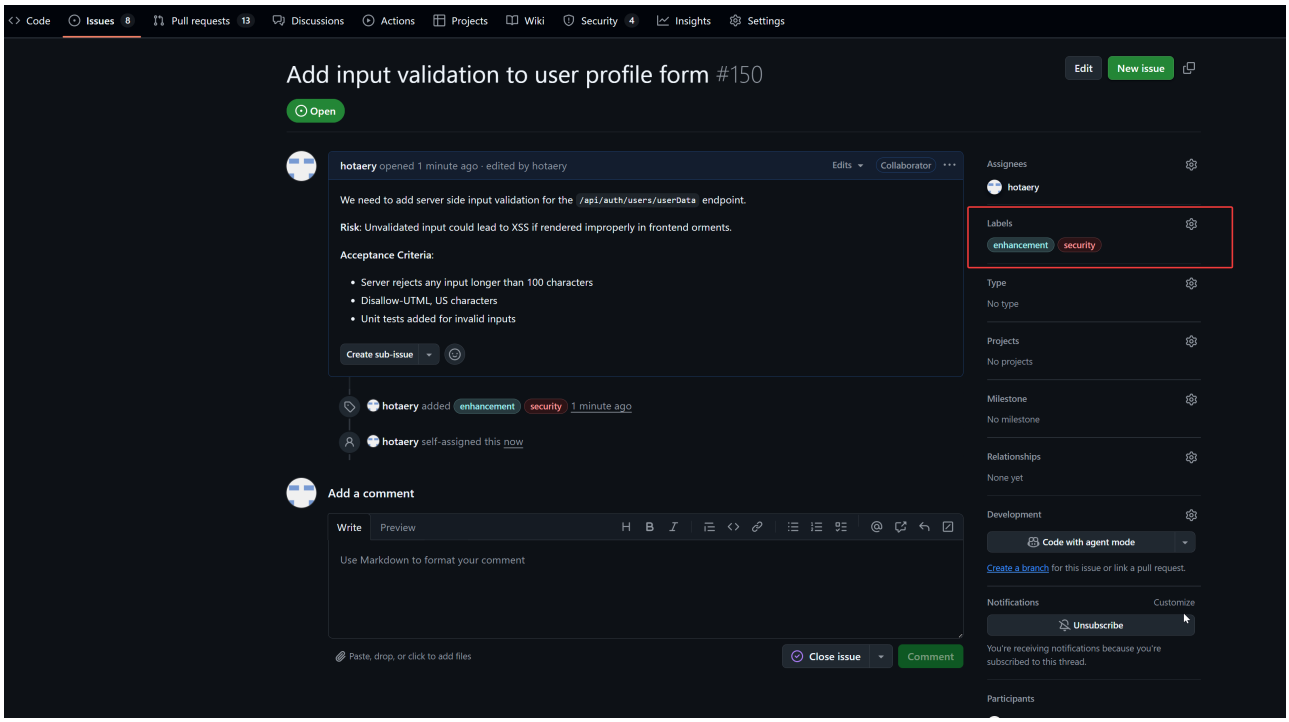
2. Requirements & Planning

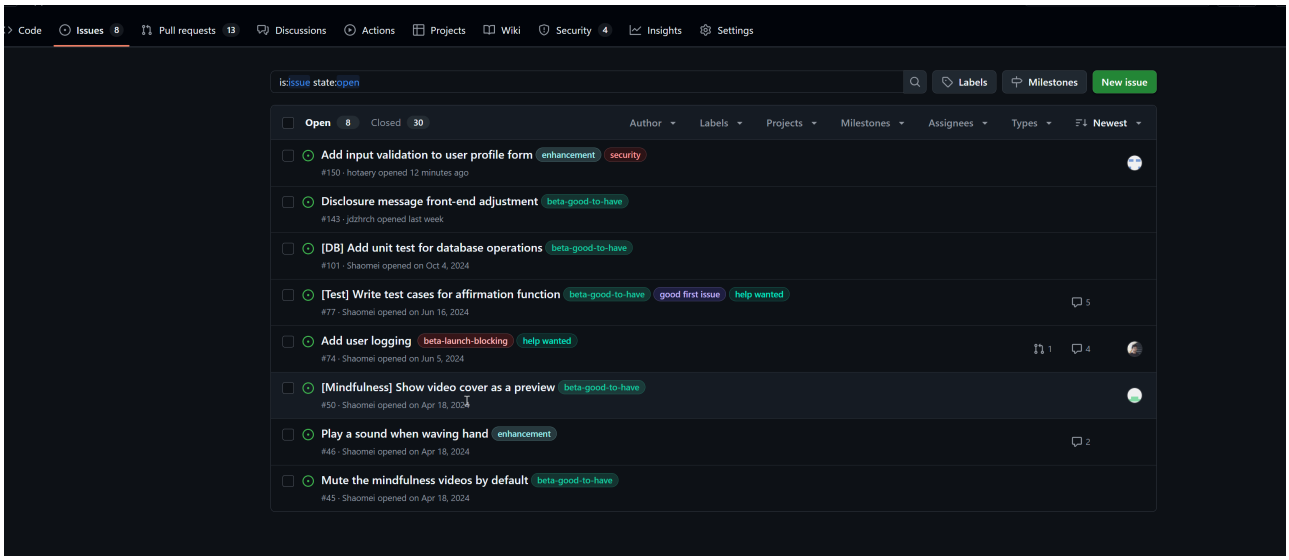
We collect requirements via GitHub Issues and discussion with stakeholders. Each issue includes:

- Feature descriptions
- Security implications
- Acceptance criteria
- Tags for risk/priority
  - `beta-block-launching` is high-priority
  - `beta-good-have` is low-priority

✅ Evidence:

- Screenshot of GitHub Issues with security labels and acceptance criteria



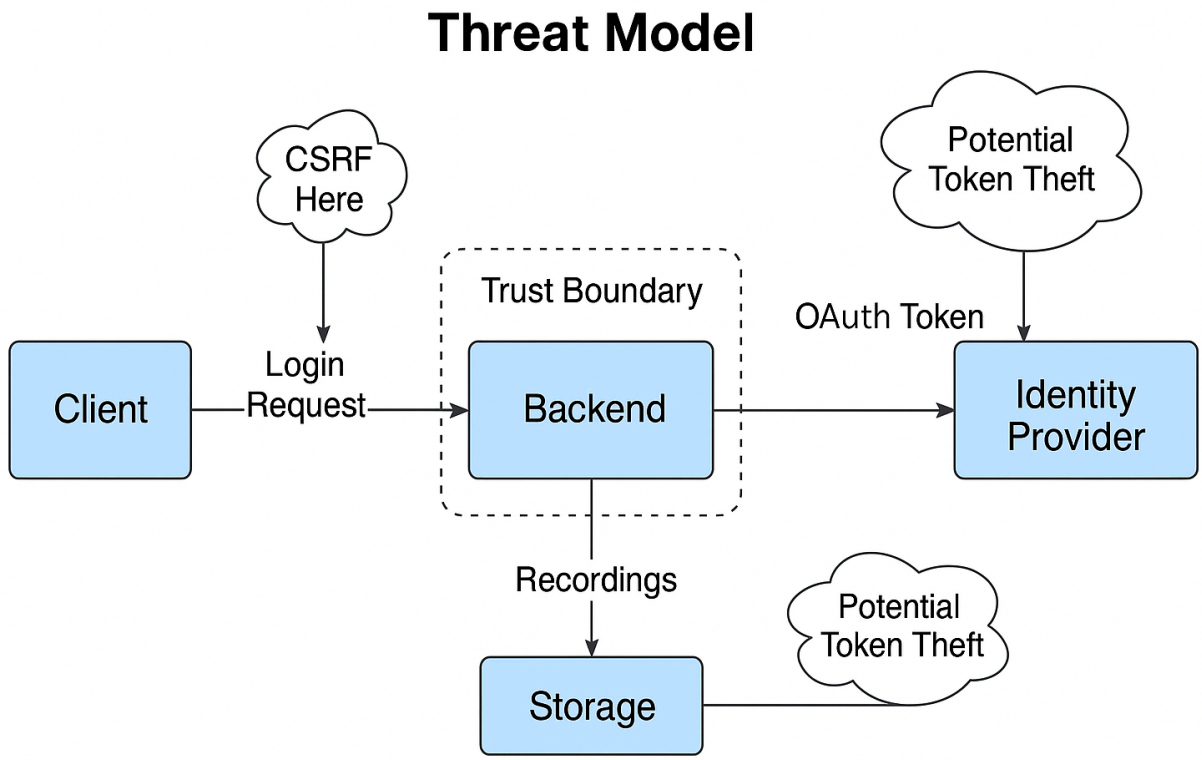


3. Design

- All features are designed using secure design principles (e.g., least privilege, input validation, rate limiting)
- Architecture diagrams are required before coding
- Threat modeling is performed using OWASP Threat Dragon
- Diagrams are reviewed and version-controlled in the repository

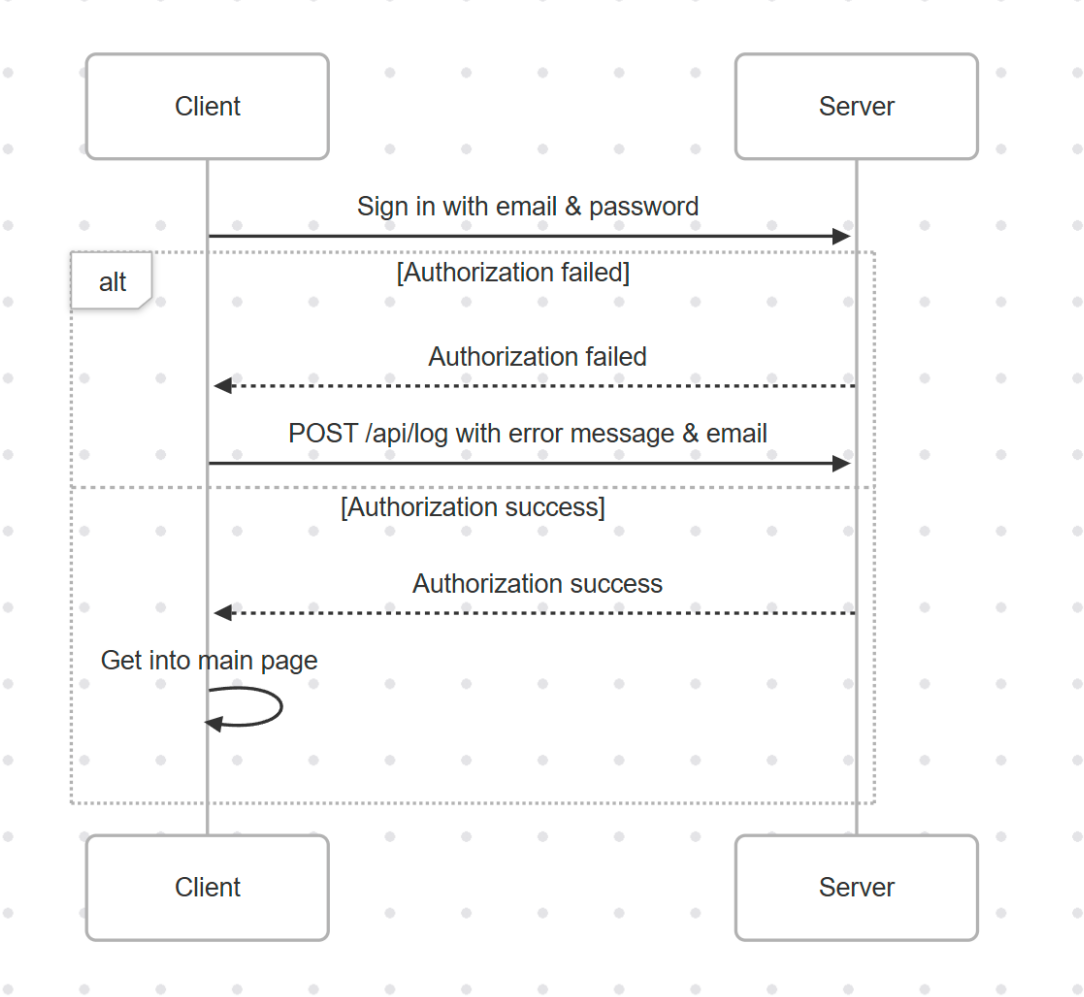
✅ Evidence:

- Threat modeling diagram screenshot

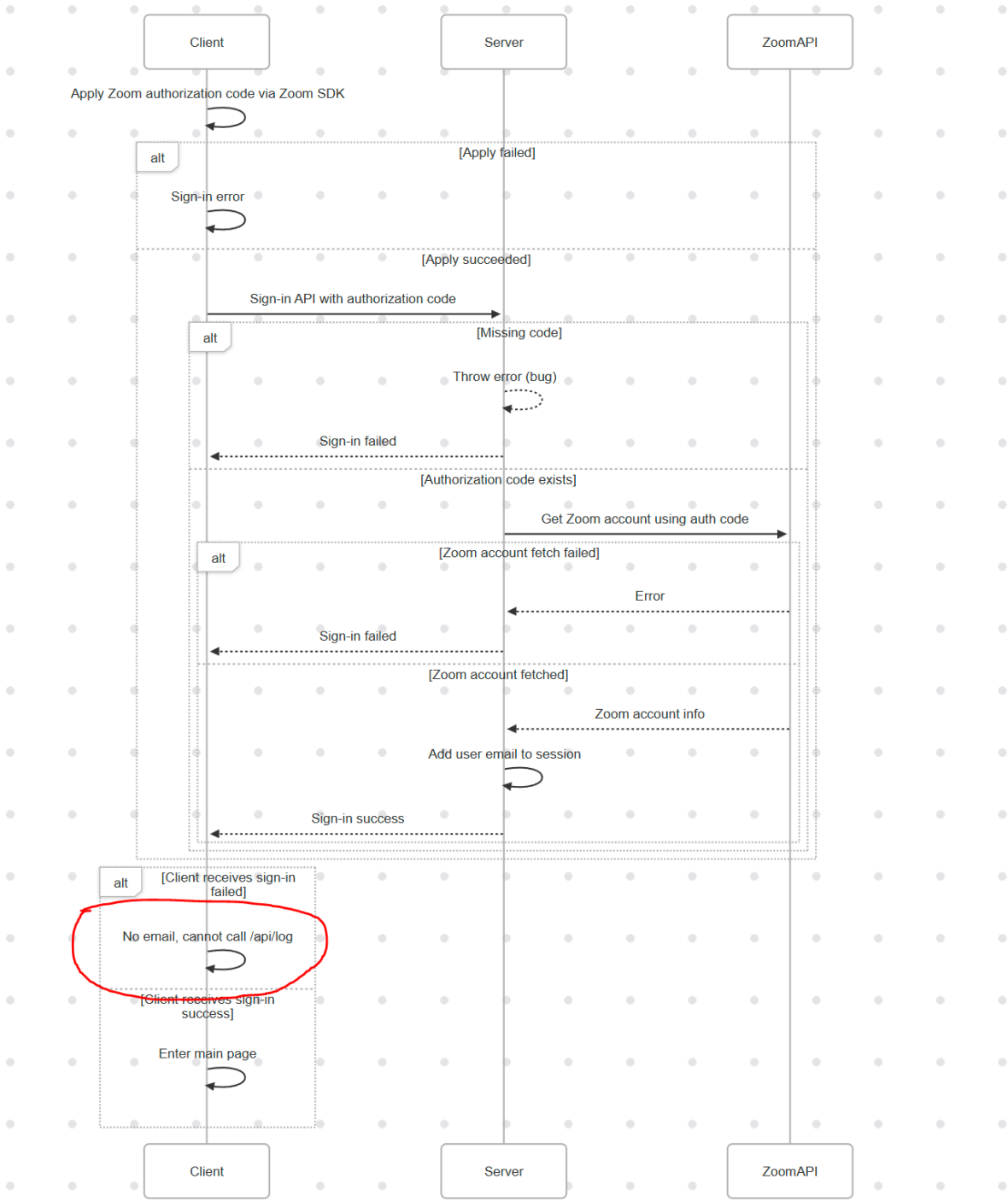


- Login via zoom OAuth flow diagram screenshot and compare with login via user/pass flow

- Login via user/pass flow:



- Login via zoom OAuth flow:

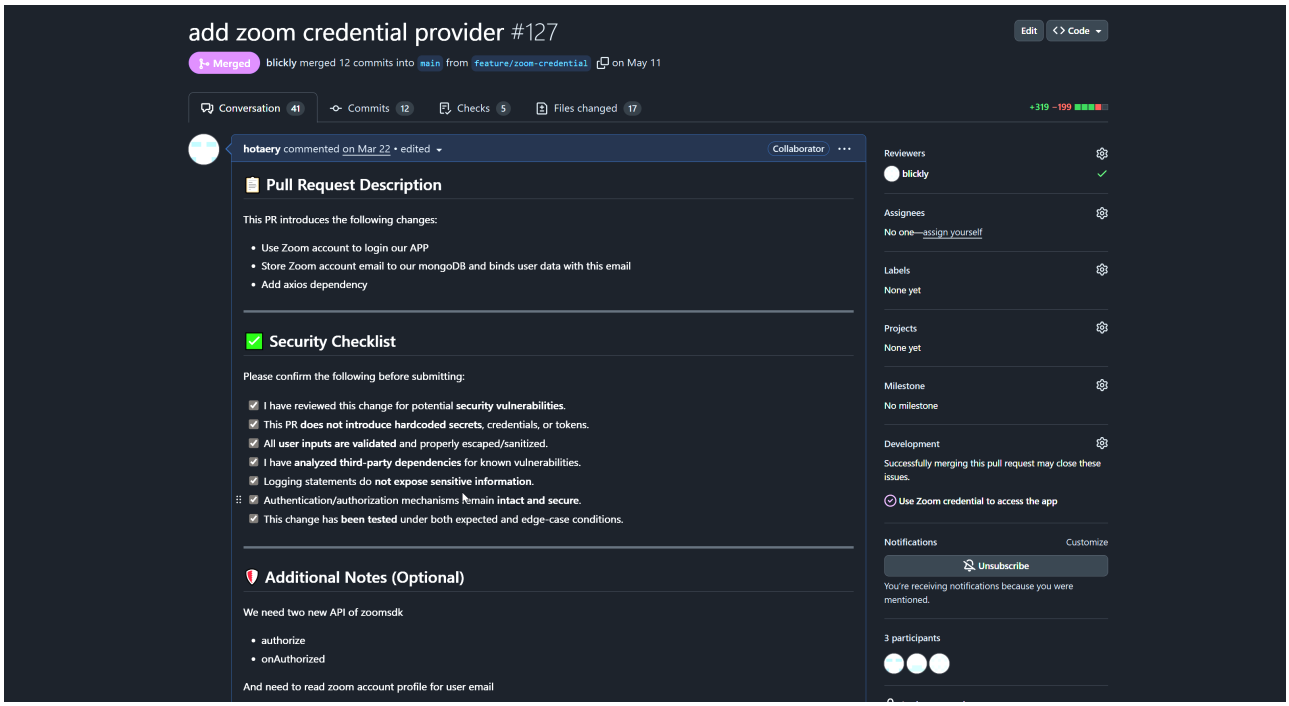


4. Implementation

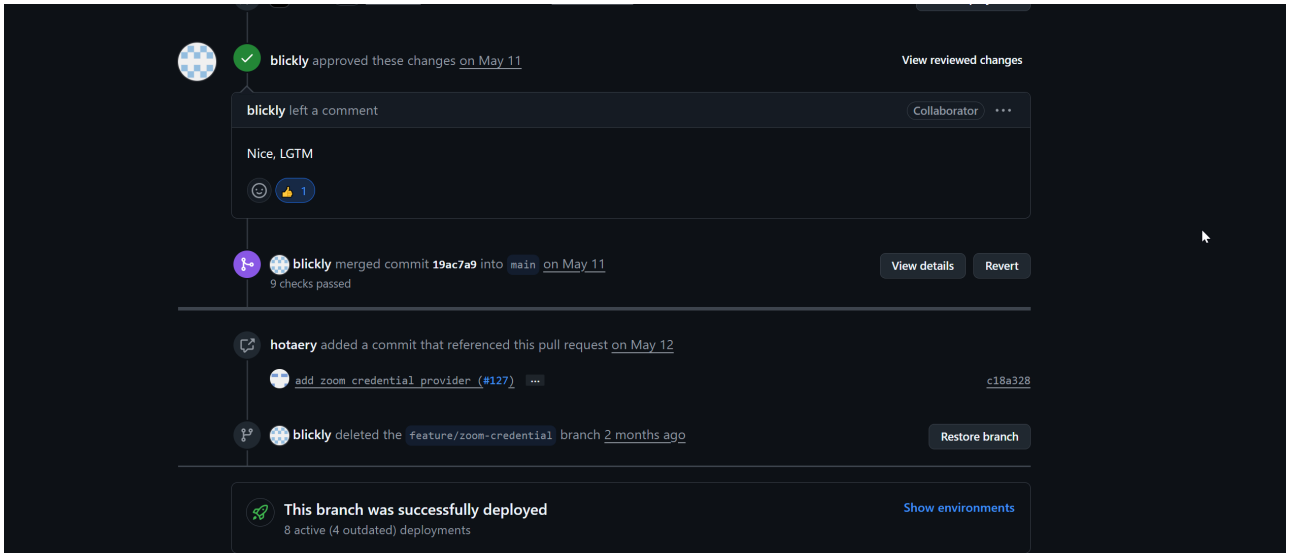
- Code is written in GitHub repos using GitHub Flow
- `main` branch is protected and requires PR approval
- Code review includes a security checklist

✅ Evidence:

- Screenshot of PR with security checklist template



- Approved PR screenshot with reviewer notes

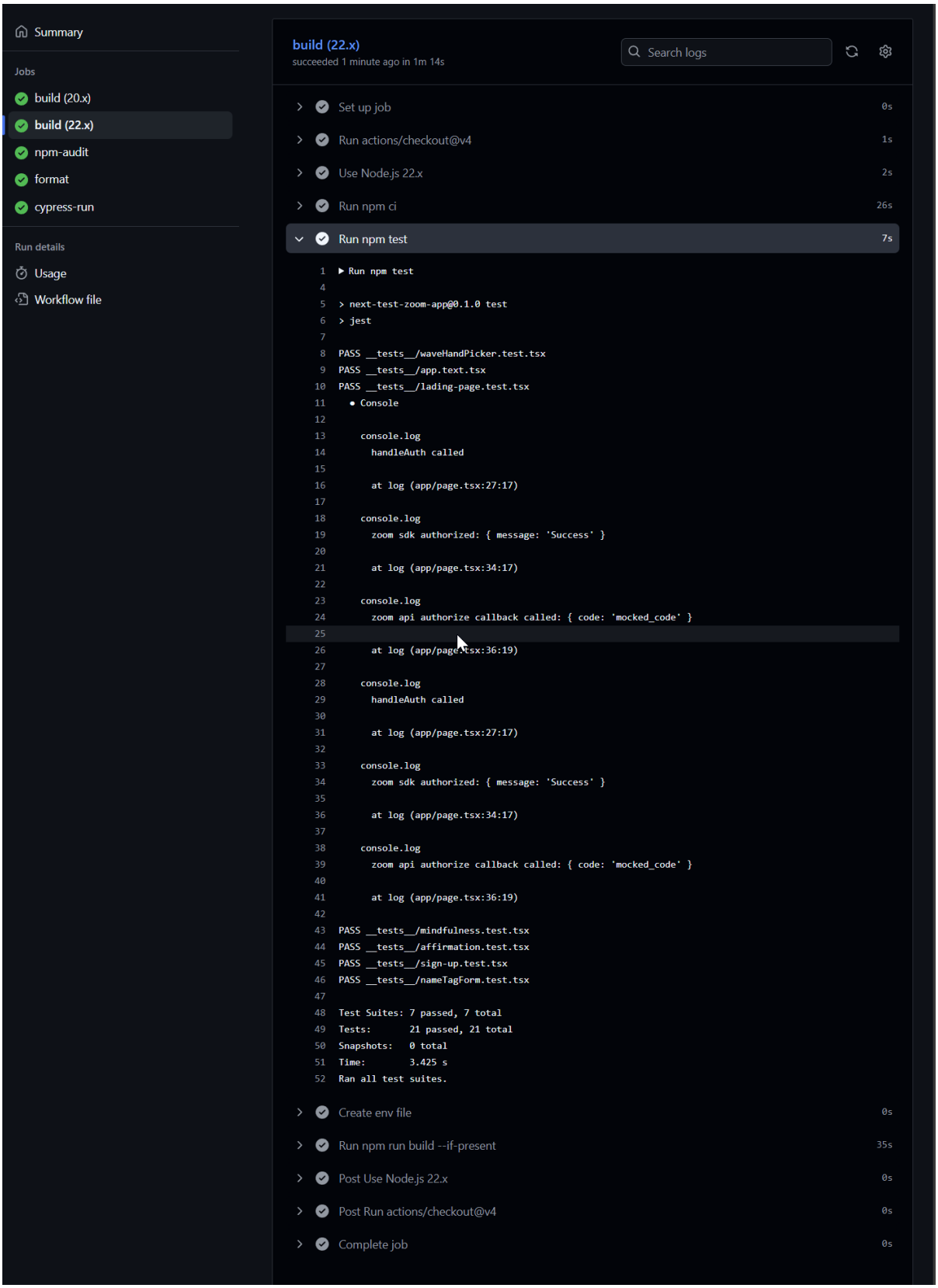


5. Testing

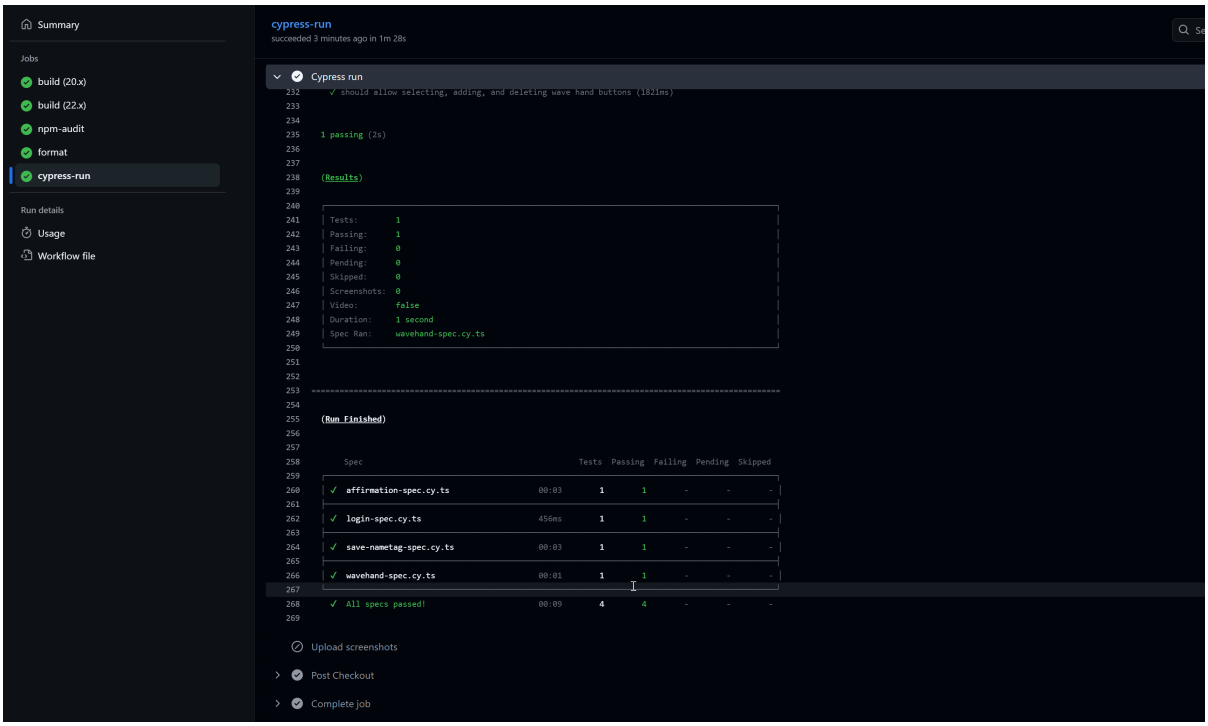
- Unit tests and Cypress E2E tests are required for all features
- GitHub Actions runs tests on each push/PR
- Security-related tests are included (e.g., input sanitization, auth)

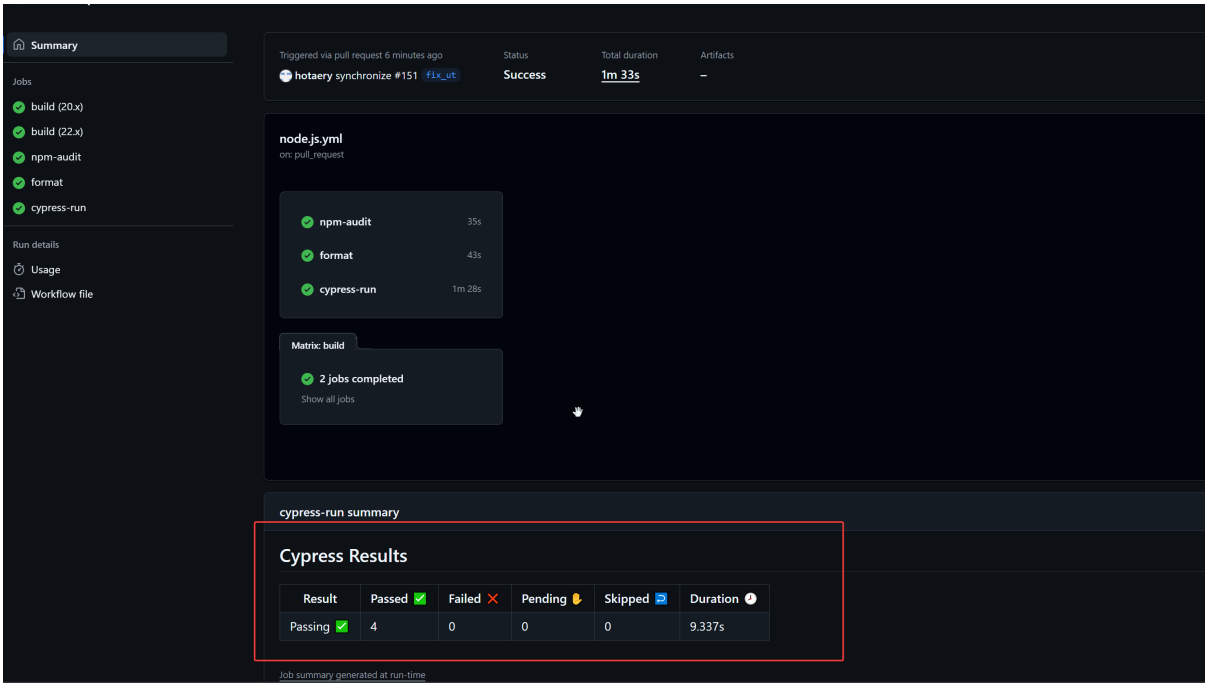
✅ Evidence:

- GitHub Actions test results
    - Unit tests



- Cypress E2E tests





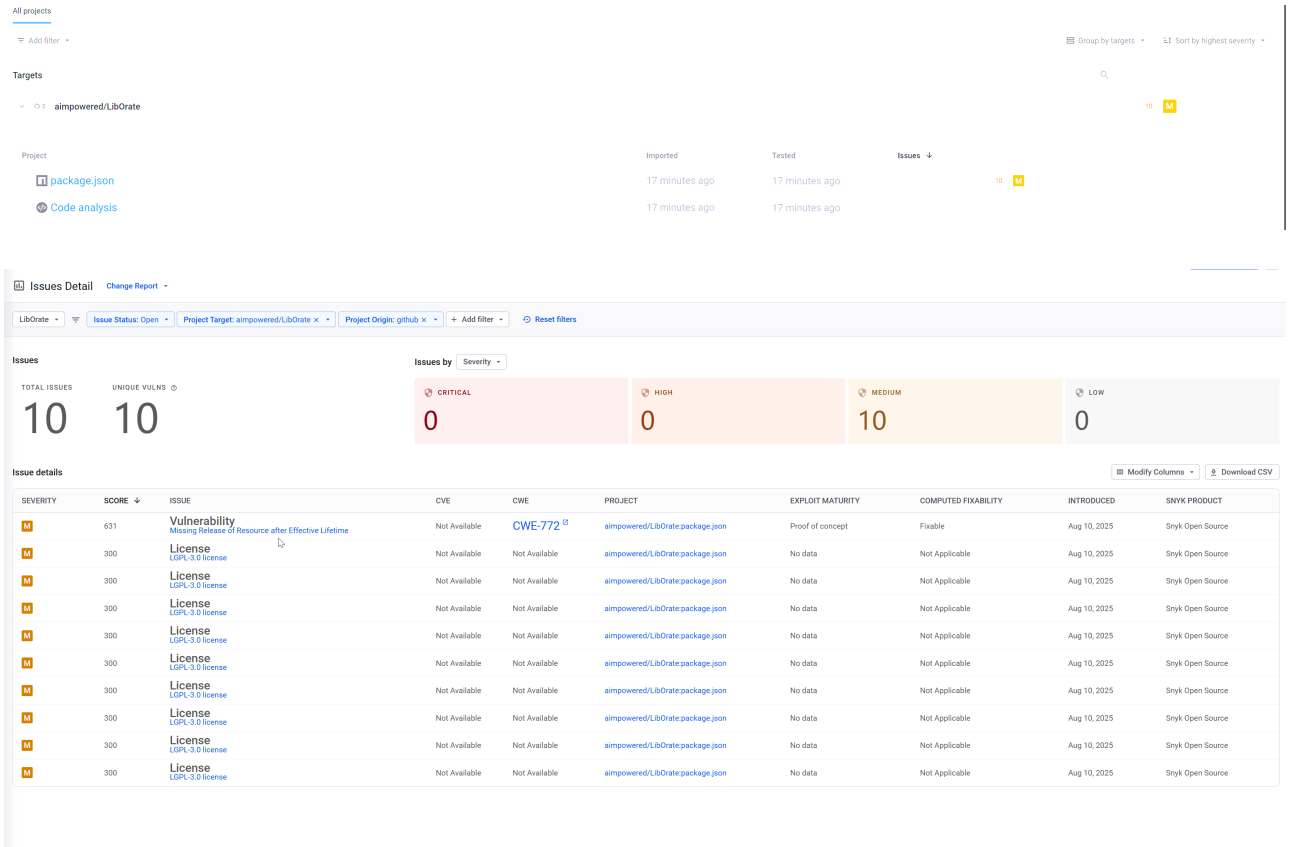
6. Security Verification (SAST, DAST, Dependency Scanning)

- Static Application Security Testing (SAST)
  - **Tools**: GitHub CodeQL, Snyk Code
  - **Scope**: Entire codebase (JavaScript/TypeScript frontend, Node.js backend)
  - **Run**: On every pull request and nightly build
  - **Findings**: Must be resolved or triaged before merge
- Dependency Scanning
  - **Tool**: GitHub Dependency Review, `npm audit`
  - Run: On every PR that changes package.json, package-lock.json, or other dependency manifests
  - Policy: Merges are blocked if high/critical CVEs are detected

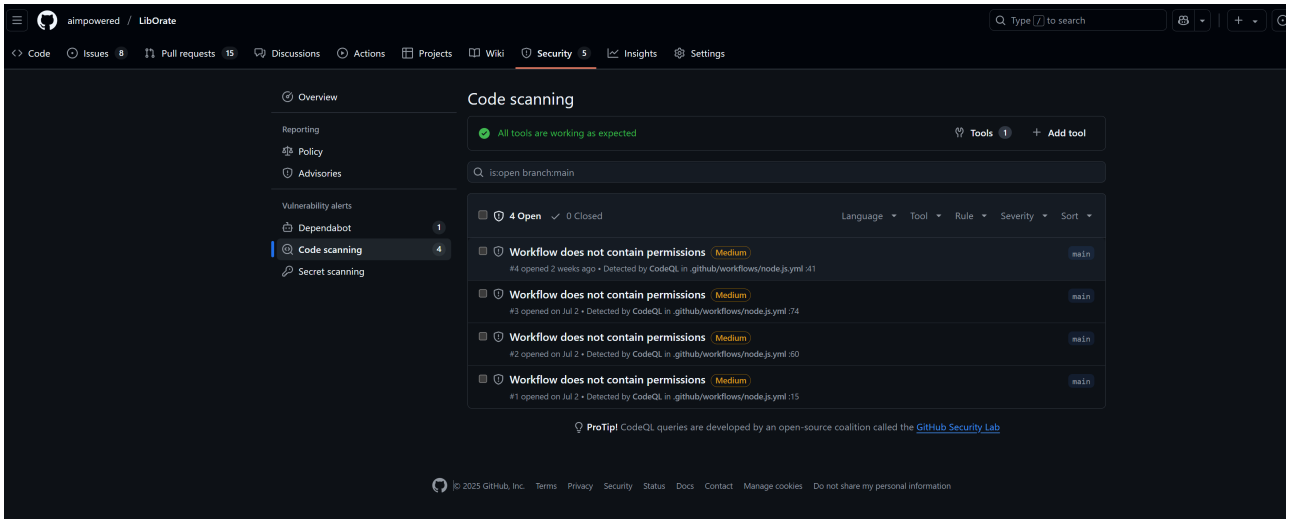
- Dynamic Application Security Testing (DAST)
  - **Tools**: OWASP ZAP
  - **Scope**: Staging environment API endpoints and web UI
  - **Run**: On staging before releases
  - **Checks**: SQL injection, XSS, CSRF, authentication bypass, open redirects

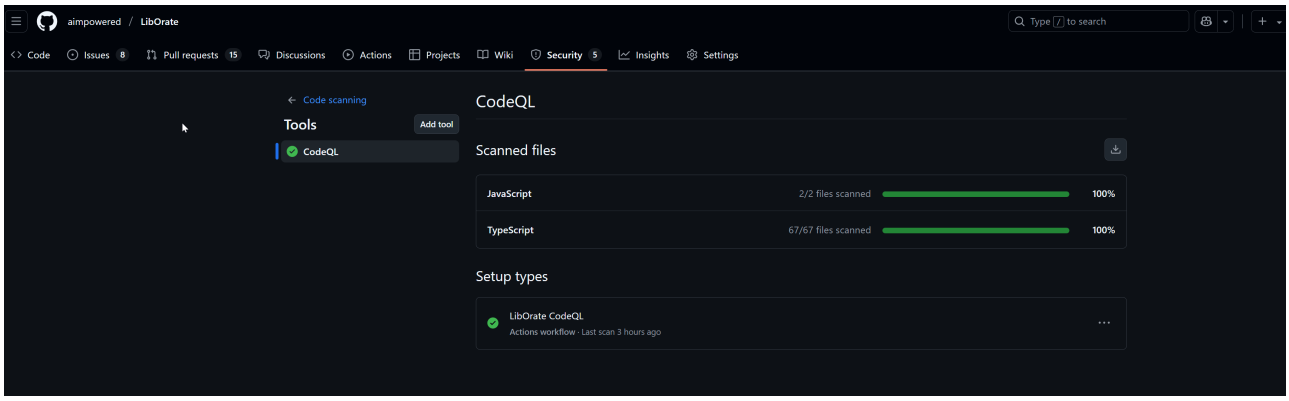
✅ Evidence:

- Snyk Code scan results showing no high/critical vulnerabilities in code. Please see `snyk_report.pdf` for full report.

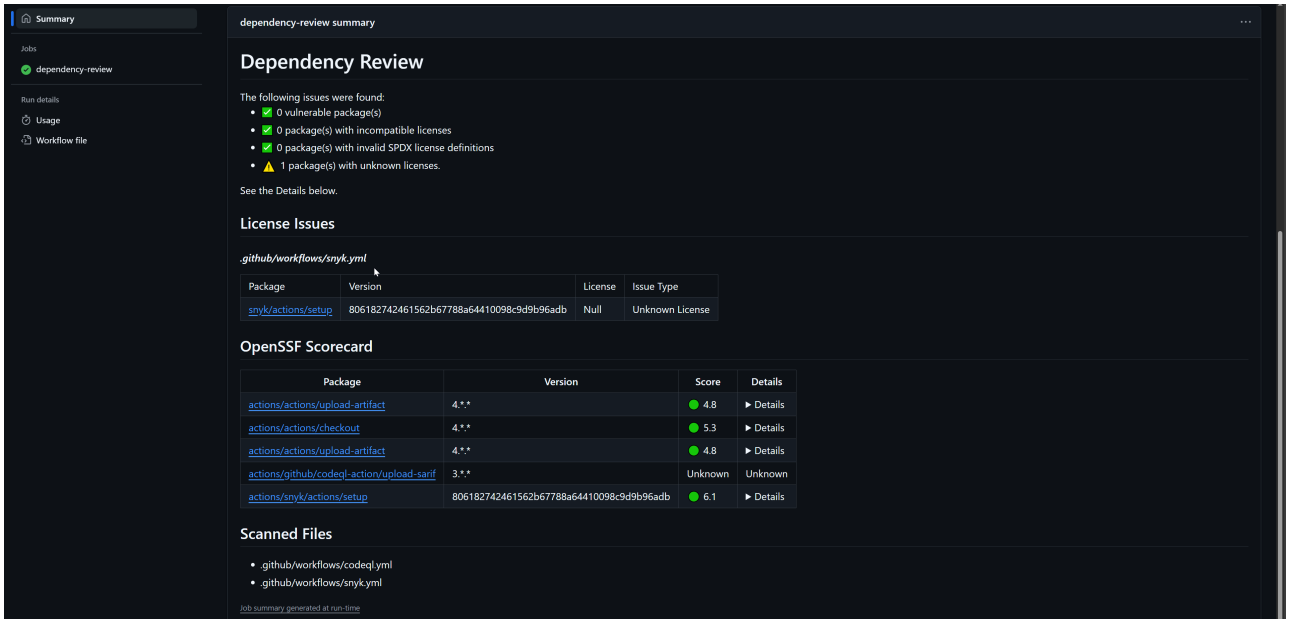


- Github CodeQL scan results showing no high/critical vulnerabilities in code

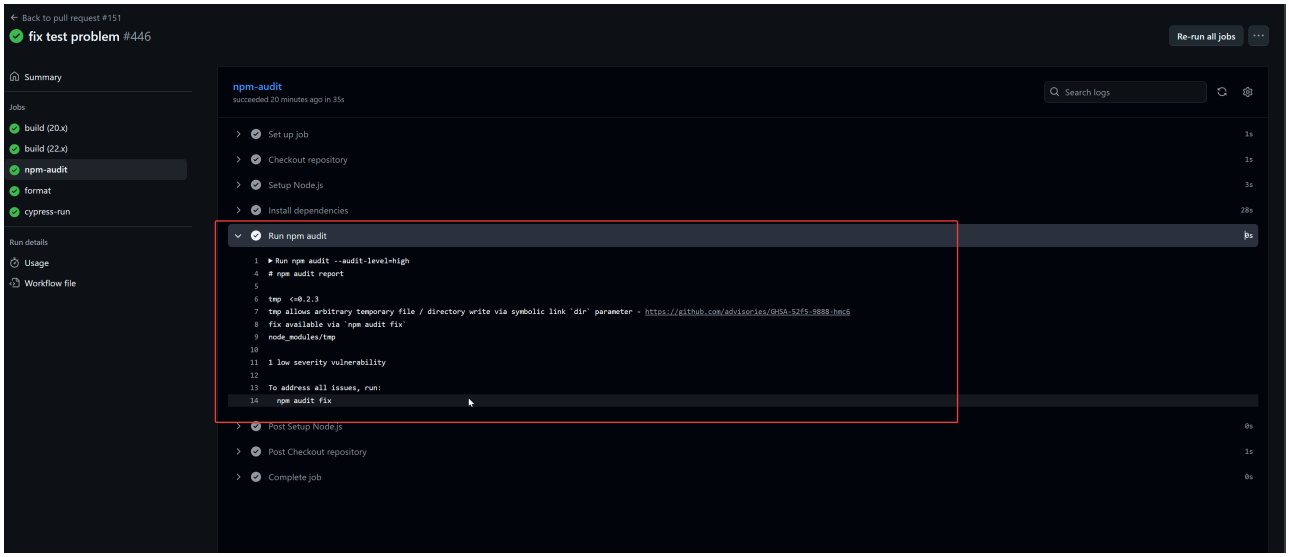




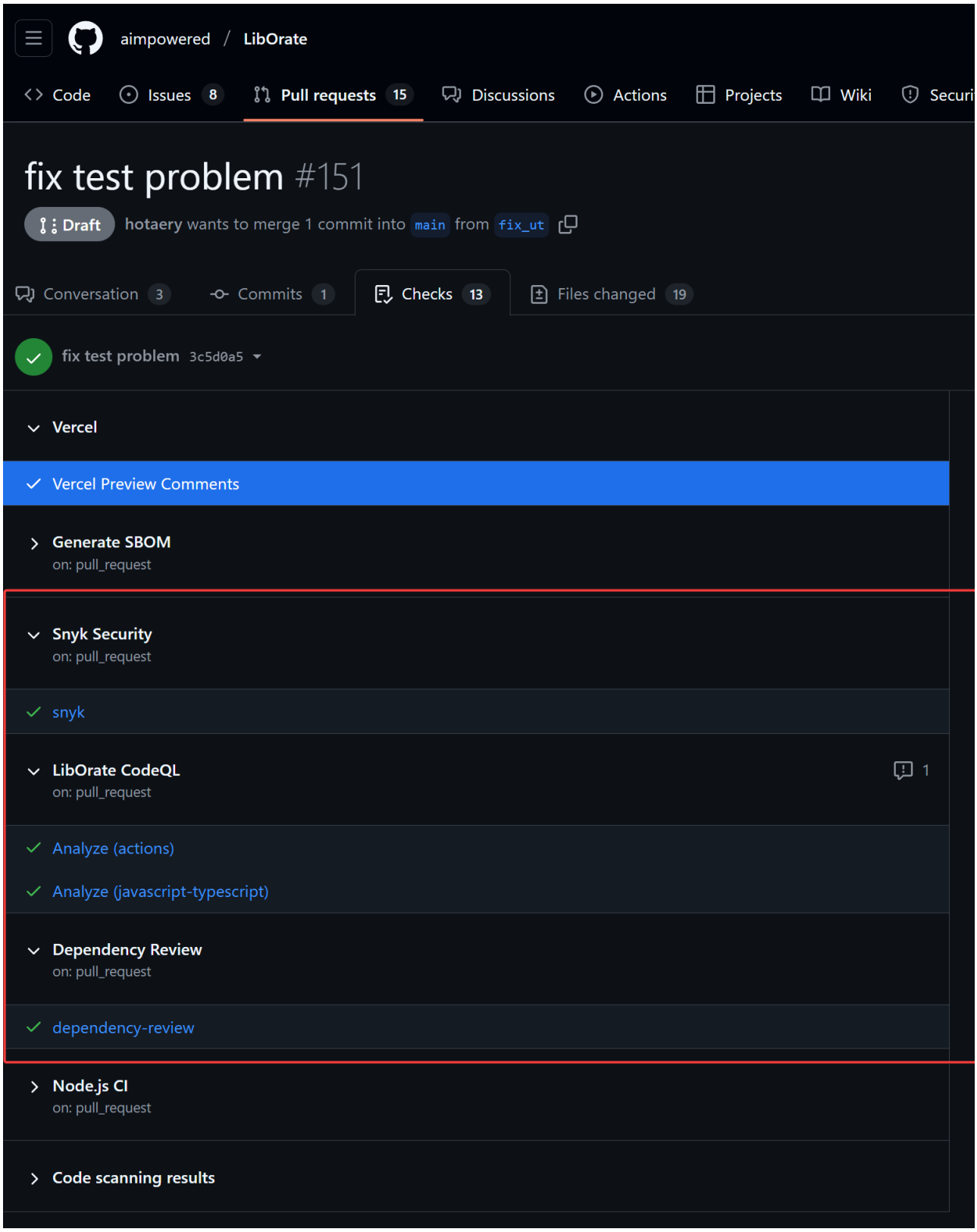
- GitHub Dependency Review report with no critical issues



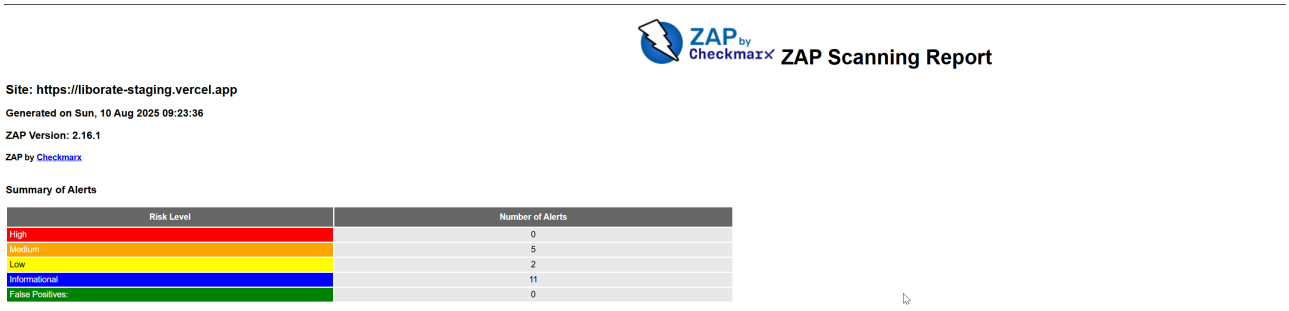
- Run `npm audit` on every PR



- Every PR is scanned for critical vulnerabilities using Snyk Code and Github CodeQL and Github Dependency Review.



- OWASP ZAP scan summary for staging. Please see `zap_report.pdf` for full report.



ZAP by Checkmarx **ZAP Scanning Report**

Site: https://liborate-staging.vercel.app

Generated on Sun, 10 Aug 2025 09:23:36

ZAP Version: 2.16.1

ZAP by Checkmarx

Summary of Alerts

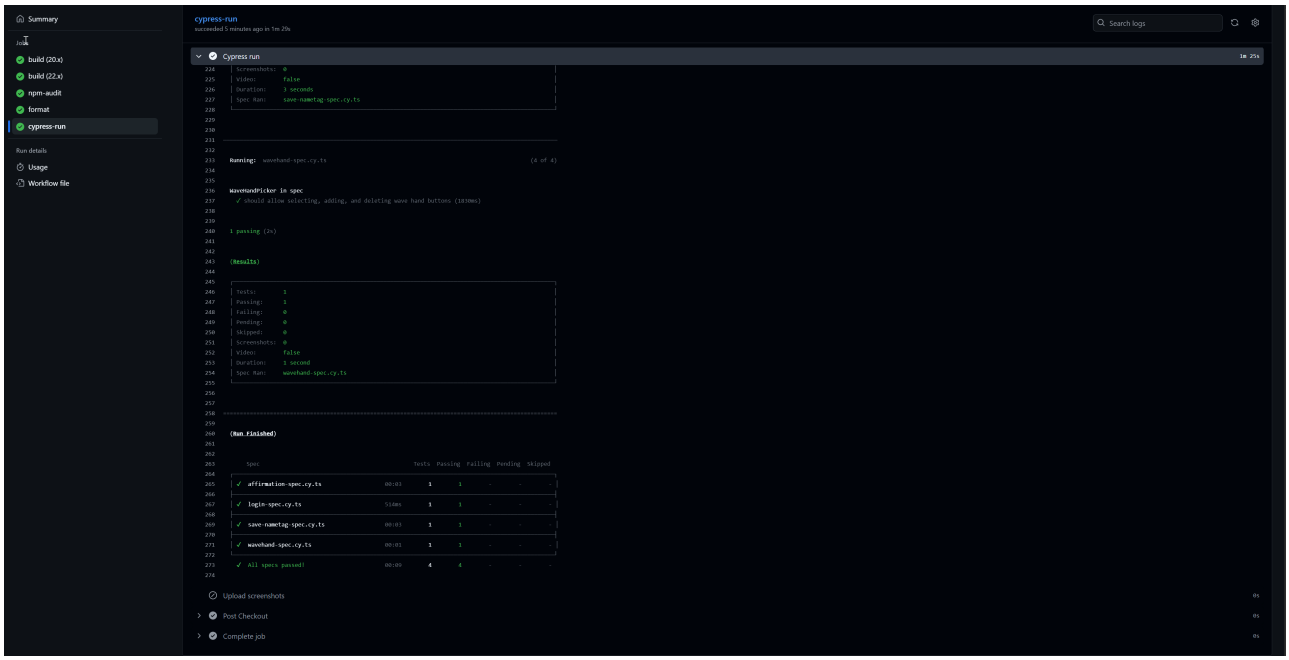
Risk Level	Number of Alerts
High	0
Medium	5
Low	2
Informational	11
False Positives:	0

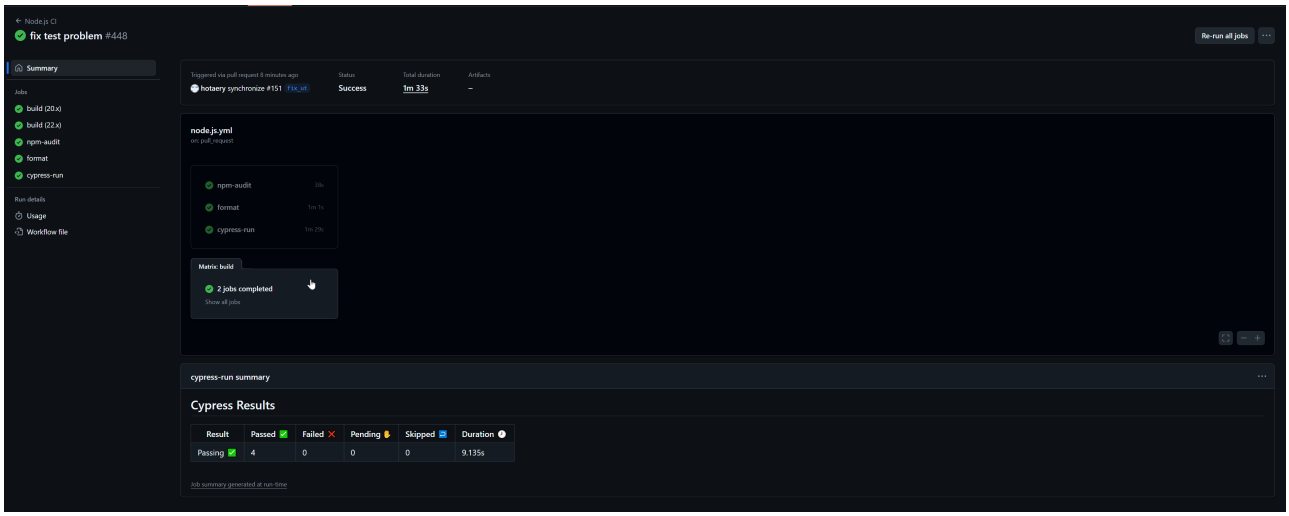
7. Deployment & Release

- Deployment to production is automated via GitHub Actions and requires:
  - Passing all tests (unit, E2E)
  - Approval from at least one security reviewer
- Staging and production use separate secrets managed in GitHub Actions Secrets
- Production builds are signed and verified before deployment
- Rollback plan documented for each release

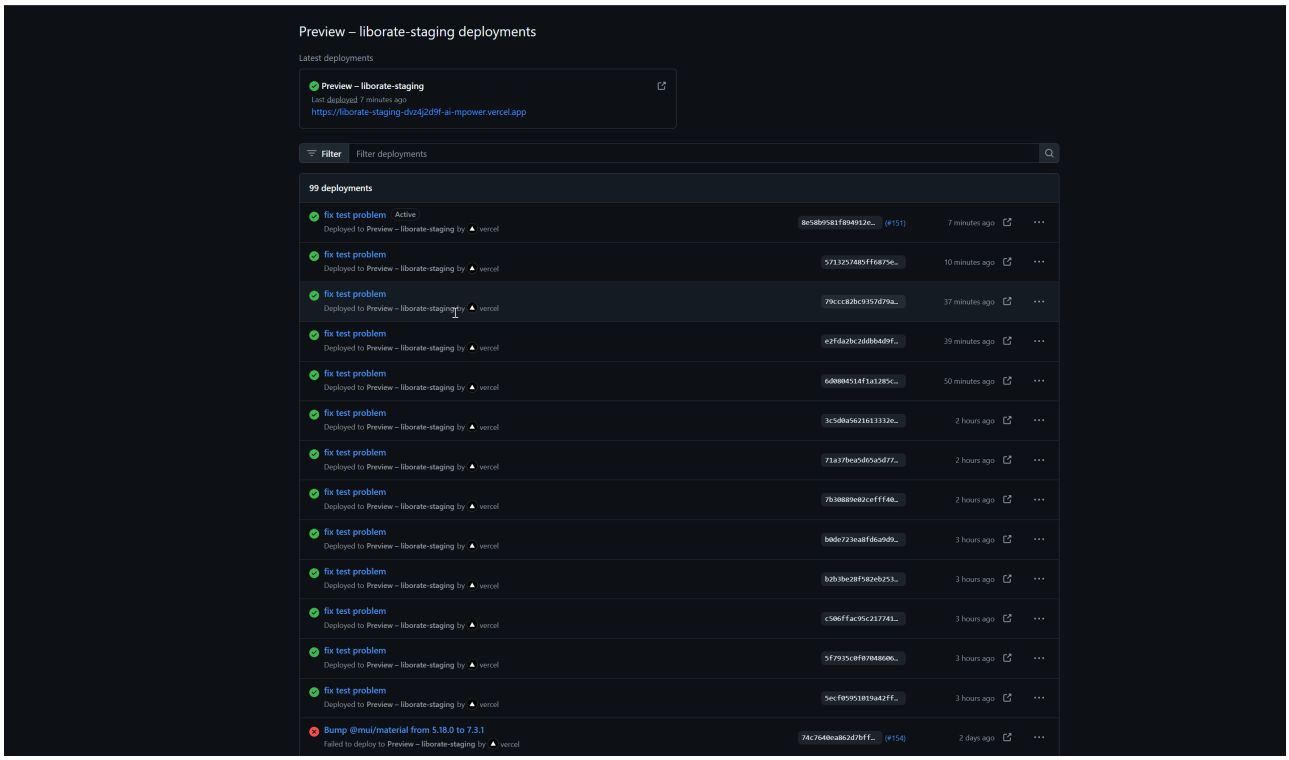
✅ Evidence:

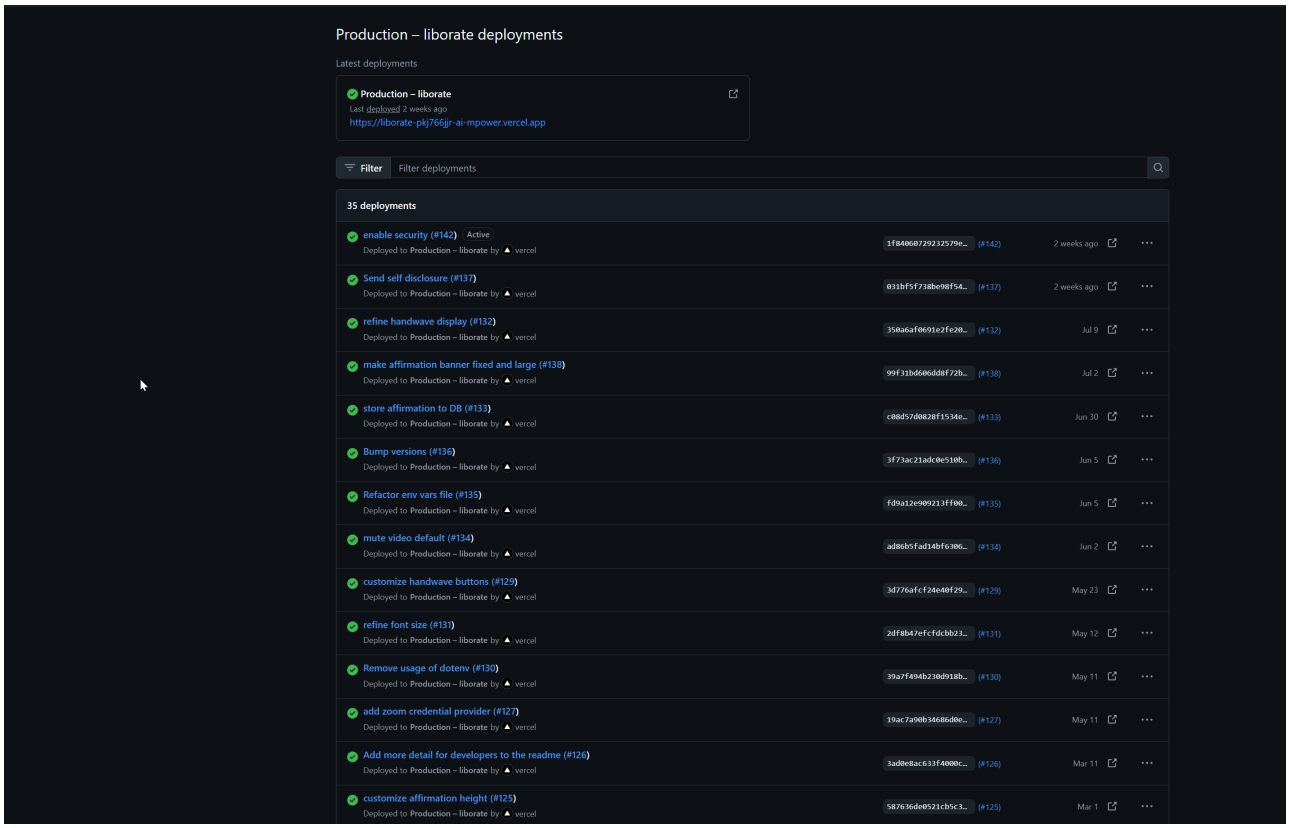
- Screenshot of GitHub Actions workflow to execute unit and E2E tests automatically





- Screenshot of GitHub Actions workflow to deploy to production




Production – liborate deployments
Latest deployments
Production – liborate
Last deployed 2 weeks ago
https://liborate-pkj766jjr-ai-mpower.vercel.app
Filter
Filter deployments
35 deployments
enable security (#142) Active
Deployed to Production – liborate by vercel
1f8406072923579e… (#142) 2 weeks ago
Send self disclosure (#137)
Deployed to Production – liborate by vercel
031bf5f738be98f54… (#137) 2 weeks ago
refine handwave display (#132)
Deployed to Production – liborate by vercel
350a6af0691e2fe20… (#132) Jul 9
make affirmation banner fixed and large (#138)
Deployed to Production – liborate by vercel
99f31bd606dd8f72b… (#138) Jul 2
store affirmation to DB (#133)
Deployed to Production – liborate by vercel
c08d57d0828f1534e… (#133) Jun 30
Bump versions (#136)
Deployed to Production – liborate by vercel
3f73ac21adc0e510b… (#136) Jun 5
Refactor env vars file (#135)
Deployed to Production – liborate by vercel
fd9a12e909213ff00… (#135) Jun 5
mute video default (#134)
Deployed to Production – liborate by vercel
ad86b5fad14bf6306… (#134) Jun 2
customize handwave buttons (#129)
Deployed to Production – liborate by vercel
3d776afcf24e40f29… (#129) May 23
refine font size (#131)
Deployed to Production – liborate by vercel
2df8b47efcfdcbb23… (#131) May 12
Remove usage of dotenv (#130)
Deployed to Production – liborate by vercel
39a7f494b230d918b… (#130) May 11
add zoom credential provider (#127)
Deployed to Production – liborate by vercel
19ac7a90b34686d0e… (#127) May 11
Add more detail for developers to the readme (#126)
Deployed to Production – liborate by vercel
3ad0e8ac633f4000c… (#126) Mar 11
customize affirmation height (#125)
Deployed to Production – liborate by vercel
587636de0521cb5c3… (#125) Mar 1